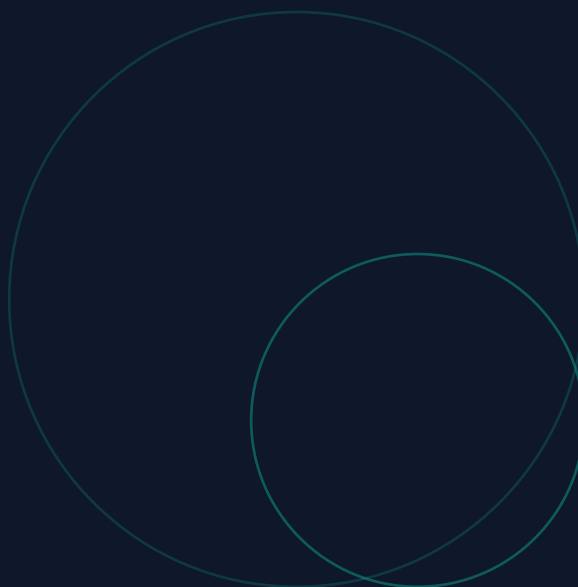


A Casa Inteira

História Completa do Ecossistema AIDD: Da Festa Infantil à Engenharia Agêntica de Produção

A jornada completa pelo Ecossistema AIDD narrada cômodo por cômodo: a metáfora viva da festa de aniversário contraposta ao rigor determinístico do código em produção.



SOBRE ESTA OBRA

Este livro corporativo apresenta a arquitetura, as leis, os fluxos e o ferramental do **Ecosystema AIDD** através de uma abordagem dupla e complementar: a narrativa lúdica da preparação de um bolo de aniversário (Na festa) e o detalhamento técnico e factual da engenharia de software determinística (Na casa).

Sem recorrer a abstrações vagas, cada cômodo da plataforma é examinado em minúcia: as 13 Leis Invioláveis, o monorepo multi-harness com fonte única em [componentes/](#), os portões de qualidade binários ([exit 0](#) ou [exit 1](#)), as worktrees efêmeras acionadas pelo Kahn DAG do despacho topológico, o Quarteto Sine Qua Non ([/docs](#), [/webhooks](#), [/mcp](#), [/docs/guia](#)), o padrão-ouro tecnológico e a economia extrema de tokens (fichas de atenção).

A obra preserva integralmente o texto autoral original no formato storytelling, complementando-o com notas técnicas de alinhamento arquitetural, caixas de contexto corporativo e matriz formal de rastreabilidade de evidências físicas.

Sumário

Prefácio — Como Ler Esta Obra e Diretrizes de Navegação	2
Capítulo 1 — A tarde inteira, sem pular cômodo	3
Capítulo 2 — O terreno	4
Capítulo 3 — O cartaz da parede — AGENTS.md	5
Os ponteiros	5
Capítulo 4 — As 13 leis	6
Capítulo 5 — Os bilhetes — rules	8
Capítulo 6 — Os botões do forno — configs	9
Capítulo 7 — A entrevista — grill	11
Capítulo 8 — A receita — spec	12
Capítulo 9 — Os pedacinhos — tickets — e o teste primeiro — tdd	13
Capítulo 10 — A planta — planner	14
Capítulo 11 — Qual cozinha — workflow	15
Capítulo 12 — As pessoas — agents	16
Capítulo 13 — Os cartões da gaveta — skills	17
Capítulo 14 — Comandos de voz e a porta da rua	19
Capítulo 15 — As máquinas — scripts	20
Capítulo 16 — O miolo compartilhado	21
Capítulo 17 — O telefone — MCP — e as quatro peças que todo bolo leva	23
O quarteto	23
Capítulo 18 — O envelope — handoff	24
Capítulo 19 — A fila — pipeline	25
Capítulo 20 — O despacho das fatias	26
Capítulo 21 — As 8 etapas de quem asse do zero	27
Capítulo 22 — O caminho de melhorar a casa que já existe	28
Onde a obra roda	28
Capítulo 23 — Os guardas — gates	29
Capítulo 24 — As campanhas — hooks	31
Capítulo 25 — A economia das fichas	32
O que gasta ficha à toa	32
O que a casa faz para gastar pouco	32
Como uma pessoa deve falar com a casa	33
O que não é economia	33
Capítulo 26 — Como se monta um projeto deste tamanho	34
O que nasce dentro do programa gerado	34
Três jeitos de uma obra andar, sem confundir	35
Capítulo 27 — O sótão — o que não é cartaz	36
Capítulo 28 — Segurança, sem fantasia	37
Capítulo 29 — Glossário — uma frase cada	38
Capítulo 30 — Mapa de pastas que importam	40
Capítulo 31 — A mesma tarde, agora com o app de lista de compras	41
Capítulo 32 — Matriz Canônica de Rastreabilidade e Evidências	42

Prefácio — Como Ler Esta Obra e Diretrizes de Navegação

Data: 21-09-2026 Onde mora: [docs/explicacoes/21-09-2026_explica-casa-completa-do-ecossistema.md](#)
De onde saiu a verdade: o cartaz [AGENTS.md](#), a memória [MEMORY.md](#), o complemento [docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md](#), o padrão [docs/protocolos/PADRAO-OURO-STACK-TECNOLOGICA.md](#), o manifesto [gates/manifesto_harnesses.json](#) e a porta [ecossistema.py](#). O que este texto não é: não é um relatório antigo, não é um plano, e não diz que a casa passou na inspeção hoje. Diz o que cada cômodo é.

Como ler. Cada cômodo tem duas falas.

- **Na festa** — a história, do jeito que uma criança acompanha.
- **Na casa** — o nome real, o arquivo real, o comando real.

A festa é sempre a mesma: você pediu um bolo de aniversário. No computador, o bolo é um programa. A casa que faz o bolo se chama Ecossistema AIDD.

DIRETRIZ METODOLÓGICA DE LEITURA

Esta obra adota o princípio pedagógico da dupla linguagem: cada conceito é apresentado primeiro na metáfora da festa (**Na festa**), permitindo apreensão intuitiva e memorização imediata, seguido rigorosamente pela anatomia real no sistema de arquivos e linhas de comando (**Na casa**). Não há concessão técnica: o rigor determinístico e as leis da engenharia agêntica são preservados integralmente.

Capítulo 1 — A tarde inteira, sem pular cômodo

Você chega numa casa grande. Na porta tem um cartaz. Em volta, bilhetes. Os botões do forno já estão marcados. Alguém abre um caderno grosso que lembra o que a casa já decidiu.

Antes de quebrar o ovo, uma pessoa pergunta tudo o que pode dar errado. Outra escreve a receita e cola na geladeira. Outra corta a receita em pedacinhos pequenos. Outra desenha a planta da cozinha.

Você escolhe uma de três cozinhas: do zero, com pedaços prontos, ou consertando um bolo que começou em outro lugar. Cada cozinha tem pessoas. Cada pessoa lê um cartão de um trabalho só. As máquinas fazem o que é sempre igual. O telefone busca o que não está na gaveta.

Ninguém grita no corredor. Passa um envelope. A fila anda. Quando o bolo tem fatias que não dependem uma da outra, cada fatia vai para uma sala separada, e a sala é jogada fora no fim. Em cada porta um guarda diz "passa" ou "volta". Se alguém tenta sair mentindo, uma campainha toca sozinha.

No fim o bolo sai com quatro coisas que não podem faltar: o livro de como usar, a campainha para o mundo de fora avisar a casa, o telefone de ferramentas, e o guia para quem não é cozinheiro. A tela é de um tipo só. O caderno de dados é de um tipo só. A conversa com quem ajuda gasta poucas fichas de atenção, porque a casa foi desenhada para não repetir a mesma história o dia inteiro.

O resto deste arquivo é cada cômodo, devagar.

Capítulo 2 — O terreno

Na festa. Não é uma cozinha. É um depósito com oito cozinhas dentro, uma porta da rua e um almoxarifado único. O que é de verdade mora no almoxarifado. O que está pendurado em cada porta é cópia.

Na casa. Isto é um monorepo: um único depósito git. A porta da rua é `python ecossistema.py`. As oito cozinhas moram em `tools/`. O almoxarifado único mora em `componentes/`. As cópias penduradas nas portas são as pastas de cada programa de conversa (os harnesses). Quem copia é a máquina `scripts/gestor_componentes.py`, chamada por `python ecossistema.py components sync`. Quem confere se a cópia ainda é igual é `python ecossistema.py components verify`.

Regra de ouro do terreno: você edita a fonte. Você não edita a cópia. Se editar a cópia, o próximo sync apaga o seu bilhete.

Cozinha	Pasta	O que faz na festa
Forge	<code>tools/aidd-forge</code>	Monta a casa nova: cartaz, guardas, botões. Comando <code>/forge</code> .
Planner	<code>tools/aidd-planner</code>	Desenha a planta e o contrato que a cozinha seguinte vai ler. Comando <code>/planner</code> .
Generator	<code>tools/aidd-generator</code>	Assa o bolo do zero, em 8 etapas. Comando <code>/generate</code> e fluxo <code>/pure</code> .
Factory	<code>tools/aidd-factory</code>	Compra pedaços prontos e encaixa. Comando <code>/factory</code> e fluxo <code>/open</code> .
Bridge	<code>tools/aidd-bridge</code>	Pega um bolo feito no Lovable, no v0 ou no Bolt e tira o cadeado. Comando <code>/bridge</code> e fluxo <code>/freedom</code> .
Master	<code>tools/aidd-master</code>	Organiza a casa em fatias. Uma fatia é um pedaço de negócio inteiro (lista, login, pagamento), não uma gaveta solta. Comando <code>/master</code> .
Enterprise	<code>tools/aidd-enterprise</code>	Carimba o que é de missão crítica e confere se o carimbo ainda bate. Comando <code>/enterprise</code> .
Ops	<code>tools/aidd-ops</code>	Leva o bolo para um computador que fica ligado o tempo todo. Comando <code>/ops</code> .

Capítulo 3 — O cartaz da parede — AGENTS.md

Na festa. O cartaz grande da entrada. Todo trabalhador lê antes de encostar na tigela. Cabe num olhar. Não é o manual da casa inteira. É a lei curta.

Na casa. O arquivo é `AGENTS.md`, na raiz. Ele manda em qualquer programa de conversa. O manual comprido, que só se abre quando precisa, é `docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md`. Abrir o manual comprido em toda conversa gasta fichas à toa. Por isso ele existe separado.

Os ponteiros

Na festa. Em cada porta da casa tem um papel pequeno: "o cartaz de verdade está na entrada".

Na casa. São arquivos que não repetem a lei. Apontam para ela.

Papel na porta	Para quem
<code>CLAUDE.md</code>	Claude Code e este tipo de conversa
<code>GEMINI.md</code>	Gemini
<code>CODEBUDDY.md</code>	CodeBuddy
<code>CODEX.md</code>	Codex
<code>MIMOCODE.md</code>	MiMo Code
<code>OPENCODE.md</code>	OpenCode
<code>QODER.md</code>	Qoder
<code>.cursor/rules/aidd.md</code>	Cursor

Mais um cartaz de memória, diferente da lei: `MEMORY.md`. A lei diz o que não pode. A memória diz o que a casa já é, o que já foi decidido, e qual comando faz o quê.

Capítulo 4 — As 13 leis

O PRINCÍPIO DO PORTÃO QUE MORDE (LEI #13)

No Ecossistema AIDD, um Quality Gate não é apenas uma rotina de verificação estática. Pela Lei 13, todo portão DEVE possuir uma suíte de teste associada que deliberadamente injeta uma condição inválida e comprova que o portão retorna `exit 1` (bloqueio). Portão que só é testado no caminho feliz (`exit 0`) é considerado fachada decorativa e reprovado na auditoria determinística.

Na festa. Treze frases que nenhum cozinheiro pode negociar. Cada frase tem um guarda com o nome dela. Se a frase não tiver guarda, o próprio cartaz é obrigado a dizer "esta frase ainda não tem guarda".

Lei	Na festa	Guarda
1	O que é mecânico, a máquina faz. Pessoa nenhuma "inventa" uma conta que um botão já resolve.	<code>gates/G_DETERMINISMO_LEI_1.py</code> , <code>G_PIPELINE_HANDOFF.py</code> , <code>G_DISPATCH_PIPELINE_VSA.py</code>
2	O guarda só diz duas coisas: passa ou volta. Não existe "mais ou menos".	<code>gates/G_SAIDA_BINARIA.py</code>
3	O que importa fica escrito em caderno, não na cabeça de quem conversou.	<code>gates/G ESTRUTURA_ESTADO.py</code> , <code>G_MIGRATION_ROT.py</code>
4	Falar pouco e no ponto. Ficha de atenção é cara.	<code>gates/G_IDIOMA_LEI_4.py</code> e a campanha <code>regra10_check.py</code>
5	Proibido deixar bilhete "termino depois" no meio do bolo. O pedaço ou existe e funciona, ou não entra.	<code>gates/G_TESTES_REAIS.py</code>
6	A mesma receita tem que funcionar em qualquer porta e em qualquer sistema.	<code>gates/G_COMPONENTE_AGNOSTICO.py</code>
7	Quem manda é a pessoa. Nada de trabalhador invisível fazendo coisa no escuro.	<code>gates/G_ZERO_HEADLESS.py</code>
8	Proibido dizer "certificado" ou "blindado" se o teste não provou.	<code>gates/G_HONESTIDADE_ROTULO.py</code>
9	Ferramenta nova só conta se foi provada num ciclo de 5 passos, com relatório.	<code>G_DISCIPLINA_TESTE_FERRAMENTA.py</code> , <code>G_ENV_ROT.py</code> , <code>G_SKILL_ROT.py</code> , <code>G_TEMPLATE_FORGE_ROT.py</code>
10	Todo bolo sai com quatro peças: livro, campanha de fora, telefone de ferramentas, guia de quem usa.	<code>G_QUARTETO_SINE_QUA_NON.py</code> , <code>G_CONTRACT_ROT.py</code>
11	A tela, o miolo e o caderno de dados têm uma receita padrão. Só muda se você pedir outra, em voz alta, na planta.	<code>gates/G_STACK_PADRAO_OURO.py</code>
12	Não tratar papel velho do sótão como se fosse o cartaz da parede.	<code>gates/G_DOCS_ROT.py</code>
13	Guarda que nunca foi pego dormindo não conta. Tem que existir um teste que quebra a regra de	<code>gates/G_PORTAO_PROVA_QUE_MORDE.py</code>

Lei	Na festa	Guarda
	propósito e vê o guarda dizer "volta".	

O guarda da lei 13 olha os outros guardas. O guarda [gates/G_LEI_DECLARA_PORTA0.py](#) olha o cartaz e cobra: toda lei aponta o seu guarda, ou assume por escrito que não tem.

Capítulo 5 — Os bilhetes — rules

Na festa. Papéis menores em volta do cartaz. Lembram um pedaço. Não substituem o cartaz.

Na casa.

- Regras de conversa deste programa: formato curto, português, sem enrolação. O arquivo que a campanha lê é [componentes/compartilhado/hooks/regra10_termos.json](#).
- Regra do Cursor: [.cursor/rules/aidd.md](#).
- Regra de nome no código novo: mecanismo genérico em inglês ([SecurityGate](#)). Coisa que só existe neste negócio em português ([materializar](#)). Está em [docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md](#), seção 4.1. Não vale para código antigo.

A forma obrigatória de uma resposta curta (a tal Regra 10):

1. Uma frase no topo dizendo o que fazer ou o que aconteceu.
2. Uma lista curta de fatos.
3. Um bloco final com uma sugestão só.

Números que a campanha usa, escritos no JSON e não na cabeça:

- texto normal: teto de 300 fichas
- texto técnico: teto de 600 fichas
- a campanha pode bloquear no máximo 2 vezes por pergunta
- resposta com menos de 30 palavras ela deixa passar

Isto vale para a conversa do dia a dia. Um documento que alguém pediu para ser completo, como este, é outra coisa: a pessoa pediu o mapa, não o bilhete.

Capítulo 6 — Os botões do forno — configs

Na festa. Temperatura, tempo, forma, lista do que pode entrar. Marcado antes. Ninguém discute no meio da massa.

Na casa. Configuração é arquivo que a máquina lê sem perguntar de novo.

Arquivo	O que o botão controla
<code>gates/manifesto_harnesses.json</code>	De qual gaveta copiar, para qual porta, em qual formato. Fonte única: componentes/ .
<code>gates/dependencias_externas.json</code>	Telefones e cartões de fora (skills e MCPs de terceiros), com o jeito de instalar e de conferir.
<code>gates/baseline_nucleo_compartilhado.json</code>	A foto do miolo compartilhado. Se master e enterprise divergirem sem estar na foto, o guarda barra.
<code>gates/termos_proibidos_marketing.json</code>	Palavras bonitas proibidas ("certificado", e afins) enquanto não houver prova.
<code>gates/allowlist_cli_help.json</code>	Exceções já revistas da ajuda da linha de comando.
<code>gates/allowlist_orfaos.json</code>	Exceções já revistas de coisa que parece solta.
<code>gates/allowlist_skipped_testes.json</code>	Testes que alguém autorizou a pular, por escrito.
<code>gates/allowlist_supply_chain.json</code>	Exceções já revistas da cadeia do que se instala.
<code>componentes/compartilhado/hooks/regra10_termos.json</code>	Tetos de ficha, forma da resposta, palavras que a campanha vigia.
<code>componentes/compartilhado/src-core/MANIFEST.json</code>	A lista do miolo compartilhado.
<code>requirements.txt</code> e <code>requirements.lock</code>	O que a casa instala para funcionar, com trava de versão e de impressão digital no lock.
<code>requirements-dev.txt</code> e <code>requirements-dev.lock</code>	O que só quem mexe na casa instala.
<code>pytest.ini</code>	Como a máquina de testes sobe.
<code>PLANO-EXECUCAO-ESTRUTURADO.json</code>	O caderno de estado da execução. A pessoa lê o caderno, não o romance da conversa antiga.
<code>opcode.jsonc</code> , <code>mimocode.jsonc</code>	Botões do OpenCode e do MiMo nesta casa.
<code>chaves/manifesto/ed25519_public.json</code>	O carimbo público. Serve para conferir que um pacote não foi trocado.
<code>keys/</code>	A chave secreta do carimbo. Não se conta, não se copia para a história, não se cola em chat.

Silêncio não é configuração. Se ninguém pediu outra tela, outra linguagem ou outro caderno, vale o padrão-ouro. Está escrito em [docs/protocolos/PADRAO-OURO-STACK-TECNOLOGICA.md](#).

Camada do bolo	O padrão, se ninguém pedir outra coisa
Tela	Next.js + TypeScript + Tailwind
Miolo	Python
Caderno	SQLite em modo WAL (várias pessoas escrevem sem rasgar a página)
Contrato da porta	OpenAPI 3.1, com estúdio em /swagger
Aviso de fora	estúdio em /webhooks
Telefone de ferramentas	estúdio em /mcp
Guia de quem usa	/docs e /docs/guia

A tela nasce do contrato da porta (`nextjs_exporter.py` no miolo, e no master o comando `export-frontend --stack nextjs`). Assim a tela e a porta não contam histórias diferentes.

Capítulo 7 — A entrevista — grill

Na festa. Antes da receita, uma pessoa senta e pergunta até doer. "E se acabar o chocolate? E se vier alguém alérgico? E se a vela não acender?" Ela não cozinha. Ela impede bolo errado.

Na casa.

- `/aidd-grill` — entrevista solta, antes de mexer em código. Cartão: `componentes/compartilhado/skills/aidd-grill/SKILL.md`.
- `/aidd-grill-docs` — a mesma entrevista, mas com o cartaz e o `MEMORY.md` abertos na mesa.

Se a fila estiver sozinha, sem pessoa para responder, a entrevista não trava a casa para sempre. Ela tem um jeito de seguir sem bloquear. O caminho normal, com pessoa, é: grill, depois spec, depois planner, depois despacho.

Capítulo 8 — A receita — spec

Na festa. O papel da geladeira. Chocolate, 8 fatias, sem amendoim, a vela acende. Também diz o que não é bolo: "não vamos fazer salgado". Cada linha dá para responder sim ou não. Não tem "ficou bonito".

Na casa. [/aidd-spec](#). Cartão: [componentes/compartilhado/skills/aidd-spec/SKILL.md](#). Sai uma especificação que a etapa seguinte consegue executar. O próximo envelope vai para o planner.

Critério binário é isto: dá para um guarda dizer passa ou volta. "Fácil de usar" não dá. "Dá para acrescentar um item, riscar, e a lista continua lá depois de fechar" dá.

Capítulo 9 — Os pedacinhos — tickets — e o teste primeiro — tdd

Na festa. A receita inteira não cabe na mão de uma pessoa. Corta-se em pedacinhos. Cada pedacinho mexe numa parte pequena da casa, numa ordem. O pedacinho 2 não começa se precisa do pedacinho 1.

Antes de assar o pedacinho, alguém prova que ele ainda não existe: faz o teste, vê falhar, aí faz o mínimo para o teste passar, aí arruma a bagunça sem quebrar o teste. Isso é o teste primeiro. Vermelho, verde, arrumar.

Na casa.

- `/aidd-tickets` — corta a spec em tarefas atômicas. Cartão: `componentes/compartilhado/skills/aidd-tickets/SKILL.md`.
- `/aidd-tdd` — vermelho, verde, arrumar, em mais de uma linguagem de teste (pytest, vitest, cargo, go). Cartão: `componentes/compartilhado/skills/aidd-tdd/SKILL.md`.
- A máquina que transforma um plano em markdown nesses pedacinhos executáveis é `scripts/compilador_tickets_plano.py`.

"Atômico" aqui quer dizer: um pedacinho, um resultado, uma prova. Se o pedacinho quebra metade da casa, ele está grande demais.

Capítulo 10 — A planta — planner

Na festa. O arquiteto. Lê a receita e entrega a planta que as três cozinhas sabem ler. Sem planta, ninguém liga o forno do fluxo grande.

Na casa. `tools/aidd-planner`. Comando: `python ecossistema.py planner` com `init`, `validate`, `export`, `audit`. O cartão é `aidd-planner-runner`. O arquivo de planta que a casa reconhece se chama `PLANNER.json` (e, na execução, `PLANO-EXECUCAO-ESTRUTURADO.json`).

O planner também monta a ordem das fatias: quem depende de quem. Se a ordem der uma volta (A espera B e B espera A), ele recusa. Isso é o compilador do despacho, no próprio planner.

Capítulo 11 — Qual cozinha — workflow

Na festa. Workflow é escolher a cozinha antes da fila andar. Nesta casa o nome oficial é fluxo. São três, e os três acabam no mesmo corredor final.

Na casa. A tríade. Quem corre os três de ponta a ponta, parando em cada envelope, é `scripts/orquestrador_sincrono.py`, pela porta:

```
python ecossistema.py run-fluxo --fluxo <pure|open|freedom> --nome <n> --slug <s> --dominio <d> --pasta <p>
```

Fluxo	Atalho	Fila
01 Do zero	<code>/pure</code>	Forge, Planner, Generator, Master, Enterprise, Ops
02 Pedacos prontos	<code>/open</code> (no Antigravity, <code>/aidd-open</code> ou <code>/factory</code> , porque <code>/open</code> lá já quer dizer abrir arquivo)	Forge, Planner, Factory, Master, Enterprise, Ops
03 Tirar o cadeado	<code>/freedom</code>	Forge, Planner, Bridge, Master, Enterprise, Ops

Os três corredores do meio são diferentes. O começo (forge + planner) e o fim (master + enterprise + ops) são os mesmos. Por isso um bolo do zero e um bolo libertado de outra ferramenta conseguem morar na mesma rua.

O guarda desta fila é `gates/G_ORQUESTRADOR_SINCRONO.py`.

Capítulo 12 — As pessoas — agents

Na festa. Uma pessoa, um trabalho. Ela não é o cartão. Ela lê o cartão e faz.

Na casa. Agent é o trabalhador daquela etapa. Os oito das cozinhas estão na tabela do terreno. Em volta deles existem trabalhadores de conversa, cada um com cartão próprio:

Pessoa	Quando entra
Grill	Antes de escrever qualquer coisa
Spec	Escreve a receita
Tickets	Corta em pedacinhos
TDD	Exige o teste antes do bolo
Diagnose	Quando algo quebrou e ninguém sabe por quê. Cinco passos: repetir o erro, olhar o mapa, uma hipótese só, provar, deixar um teste que impede o erro de voltar. / <code>aidd-diagnose</code>
Handoff	Quando a conversa vai trocar de pessoa. / <code>aidd-handoff</code>
Melhoria	Olha o que já existe e dá nota com prova. / <code>melhoria</code>
Plano	Transforma a nota num plano de obra. / <code>plan</code>
Orchestrate	Escolhe onde a obra roda. / <code>orchestrate</code>
Livro	Escreve o livro da obra a partir do que realmente foi feito, não do que alguém prometeu. / <code>aidd-livro-texto</code> e <code>python ecossistema.py livro</code>

Lei 7 de novo, porque é fácil esquecer: a pessoa de verdade continua no controle. Trabalhador invisível, no escuro, sem você ver, é proibido. Existe uma flag perigosa, `--dangerously-force-headless`, exatamente para deixar claro que alguém está pedindo a exceção. O normal é não usar.

Se você aperta parar, a casa marca a etapa como falha, joga fora a sala temporária e solta o cadeado. Não deixa sala fantasma.

Capítulo 13 — Os cartões da gaveta — skills

Na festa. O cartão ensina um trabalho. Não cozinha sozinho. A pessoa lê o cartão para não inventar o jeito.

Na casa. Cada cartão é uma pasta com um `SKILL.md`. A gaveta de verdade é `componentes/compartilhado/skills/<nome>/`. As portas (`.claude/skills`, `.agents/skills`, `.cursor/skills`, `.opencode/skills`, `.mimocode/skills`, `.gemini/extensions/<nome>/`, `.codebuddy/`) recebem cópia. O Gemini não lê o cartão solto: cada cartão vira uma extensão, com um `gemini-extension.json` gerado na hora.

Cartões da própria casa (não os de terceiros):

Cartão	Uma frase
<code>aidd-forge-runner</code>	Sobe governança num projeto.
<code>aidd-planner-runner</code>	Gera e confere a planta.
<code>aidd-generator-runner</code>	Dispara as 8 etapas do zero.
<code>aidd-factory-runner</code>	Encaixa motores prontos.
<code>aidd-bridge-runner</code>	Operações miúdas do bridge: escanear, converter banco, juntar, empacotar.
<code>aidd-master-runner</code>	Abre uma fatia nova.
<code>aidd-enterprise-runner</code>	Injeta peça carimbada.
<code>aidd-ops-runner</code>	Sobe a infra.
<code>aidd-pure, fluxo-01-runner</code>	Fluxo 01 inteiro.
<code>aidd-open, open, fluxo-02-runner</code>	Fluxo 02 inteiro.
<code>aidd-freedom, freedom, fluxo-03-runner</code>	Fluxo 03 inteiro.
<code>aidd-orchestrator-runner</code>	O maestro que corre qualquer um dos três.
<code>aidd-pipeline-runner</code>	A fila com salas temporárias.
<code>aidd-dispatch-runner</code>	O despacho das fatias na ordem certa.
<code>aidd-grill, aidd-grill-docs</code>	A entrevista.
<code>aidd-spec</code>	A receita.
<code>aidd-tickets</code>	Os pedacinhos.
<code>aidd-tdd</code>	O teste primeiro.
<code>aidd-diagnose</code>	Achar o que quebrou.
<code>aidd-handoff</code>	O envelope da conversa.
<code>aidd-melhoria, melhoria</code>	Nota com prova.
<code>aidd-plan, aidd-planos, plan, planos-auditoria-runner</code>	O plano de obra.
<code>aidd-orchestrate, aidd-orca, orca-plan-orchestrator</code>	Onde e como a obra roda.
<code>aidd-componentes, componentes-runner</code>	Cria e sincroniza cartões entre as portas.
<code>aidd-skills, skill-creator-runner</code>	Cria cartão novo.
<code>aidd-dependencias, dependencia-runner</code>	Instala e confere o que veio de fora.
<code>aidd-mcp, mcp-creator-runner</code>	Abre um telefone novo.
<code>aidd-livro-texto</code>	O livro da obra.

Regra de um dono só: um comando de voz tem um cartão dono. Dois cartões respondendo ao mesmo comando, com regras opostas, já fizeram a casa construir mesa dentro de mesa. Não se repete.

O guarda que confere se o cartão aponta para arquivo que existe é [gates/G_SKILL_ROT.py](#).

Cartões de fora (impeccable, code-review-graph, e outros) não nascem nesta gaveta. Entram pelo manifesto [gates/dependencias_externas.json](#), pelo comando `python ecossistema.py dependencia`.

Capítulo 14 — Comandos de voz e a porta da rua

Na festa. Você pode falar "pure" ou apertar o botão da porta. Os dois chamam a mesma cozinha. Fingir que não ouviu é proibido.

Na casa. O comando de voz mora em `componentes/compartilhado/comandos/<nome>.md` e é copiado para a pasta `commands` de cada porta. A porta que sempre existe, com ou sem programa de conversa, é: `python ecossistema.py <comando>`

Você diz	A porta faz
<code>/pure, /open, /freedom</code>	O fluxo inteiro
<code>/forge, /planner, /generate, /factory, /bridge, /master, /enterprise, /ops</code>	Uma cozinha só
<code>/run-plan <plano></code>	Compila o plano e corre a fila
<code>/pipeline</code>	Corre a fila a partir de um envelope já pronto
<code>/dispatch, /aidd-dispatch</code>	Despacha as fatias
<code>/melhoria, /plan, /orchestrate</code>	Olhar, planejar, executar obra na casa que já existe
<code>audit</code>	Chama os guardas
<code>components sync\ verify</code>	Copia ou confere os cartões
<code>dependencia bootstrap\ add-skill\ add-mcp\ list\ verify</code>	O que veio de fora
<code>harness status\ clean</code>	Vê e limpa o lixo de memória dos programas de conversa
<code>preflight-host</code>	Confere se Git, Node, Docker e os verificadores de receita de container estão na máquina
<code>status</code>	Diz o que está ligado
<code>livro <pasta></code>	Gera o livro da obra e passa no guarda de evidência
<code>help</code>	Lê o letreiro

O guarda que confere se a ajuda da porta não mente sobre os botões é `gates/G_CLI_HELP_CONSISTENCIA.py`.

Capítulo 15 — As máquinas — scripts

Na festa. Lavar a tigela do mesmo jeito, toda vez. Sem opinião. Sem "acho que hoje eu faço diferente".

Na casa. Pasta `scripts/`. Máquina não chama modelo de linguagem para contar, copiar, validar ou ordenar. Isso é a lei 1.

Máquina	O gesto
<code>ecossistema.py</code>	A porta da rua. Não está em <code>scripts/</code> . Está na raiz. Encaminha todo o resto.
<code>scripts/orquestrador_sincrono.py</code>	Corre um fluxo inteiro e confere o envelope em cada passagem.
<code>scripts/compilador_tickets_plano.py</code>	Lê o plano em markdown e escreve o envelope da fila.
<code>scripts/gestor_componentes.py</code>	Copia e confere cartões, comandos, telefones e campanhas.
<code>scripts/gestor_dependencias.py</code>	Instala e confere o que é de fora.
<code>scripts/gerenciador_planos.py</code>	Abre, nota, aprova e move planos.
<code>scripts/gerenciador_melhorias.py</code>	Abre o relatório de nota.
<code>scripts/atualizar_index_planos.py</code>	Reescreve o índice dos planos pelo estado real, não pelo que alguém disse.
<code>scripts/preflight_host.py</code>	Olha se as ferramentas da máquina existem.
<code>scripts/harness_hygiene.py</code>	Limpa cache e banco inchado dos programas de conversa.
<code>scripts/gerador_livro_projeto.py</code>	Monta o livro a partir de artefato que está no disco.
<code>scripts/gerar_chave_manifesto.py</code>	Gera o par de carimbo.
<code>scripts/faz_commit.py</code>	O caminho curto de registrar a mudança.
<code>tools/aidd-master/scripts/orchestrator_pipeline.py</code>	A fila dentro de salas temporárias, com a barreira no fim.
<code>tools/aidd-master/scripts/dispatch_pipeline.py</code>	O despacho das fatias.
<code>tools/aidd-master/scripts/engine_router.py</code>	Escolhe generator, factory ou bridge e cobra o quarteto.
<code>tools/aidd-master/scripts/vsa_join_barrier.py</code>	Só junta de volta o que a fatia realmente tinha licença de mexer.

Sala temporária, na prática: uma worktree git. Uma cópia de trabalho isolada, em `.worktrees/<nome-da-fatia>`, que nasce para uma tarefa e é apagada no fim, mesmo se a tarefa quebrar. O `try-finally` é a promessa "apague a sala aconteça o que acontecer".

Capítulo 16 — O miolo compartilhado

Na festa. Painéis que todas as cozinhas usam. Se cada cozinha trouxesse a sua panela diferente, o bolo mudava de gosto no meio do corredor.

Na casa. [componentes/compartilhado/src-core/](#). O master e o enterprise compartilham linhagem deste miolo. Se um lado mudar e o outro não, sem a mudança estar na foto, [gates/G_DRIFT_NUCLEO_COMPARTILHADO.py](#) barra.

Peça	Na festa
result.py	O jeito único de dizer "deu certo" ou "deu errado", sem meio termo.
escritor_atomico.py	Escreve o arquivo inteiro ou não escreve. Não deixa papel pela metade.
database.py , database_adapter.py	O caderno.
openapi.py , swagger.html	O contrato da porta e a página para ler o contrato.
webhooks.py , webhook_studio.html	A campanha que o mundo de fora toca.
mcp_server.py , mcp_studio.html	O telefone e a página do telefone.
nextjs_exporter.py	Costura a tela a partir do contrato.
caveman_protocol.py	O jeito de falar curto para gastar menos ficha. Três momentos: o que entra comprimido, o pensamento curto, a resposta em português.
security.py , token_revocation.py , assinatura_manifesto.py , package_verifier.py	Fechadura, crachá cancelado, carimbo, conferência de pacote.
events.py , outbox_worker.py , transaction_log.py , cqrs.py	Recado que não se perde, fila de recados, diário do que mudou, separar "perguntar" de "mudar".
circuit_breaker.py	Se o telefone de fora está mudo, para de ligar toda hora.
sandbox_runner.py	Roda coisa nova numa caixa, não na cozinha principal.
intent_router.py	Ouve o que você pediu e aponta a cozinha certa.
sincronizador_harness.py , materializador.py , sync.py	Tiram a peça da gaveta e colocam na porta certa.
subagent_engine.py	O motor de trabalhadores ajudantes, debaixo da lei de que você continua vendo.
logs.py , metrics.py , opentelemetry.py	O diário e os relógios.
jobs.py , local_first.py , fleet_discovery.py	Tarefa de fundo, funcionar primeiro na sua máquina, achar as outras casas.
design_catalog.py	O catálogo do visual.
fuzzing.py	Cutuca a porta com entrada esquisita para ver se ela quebra.
MANIFEST.json	A lista do que este miolo afirma conter.

Na raiz, três peças irmãs de atenção:

Peça	Na festa
core/context_slicer.py	Em vez de ler o livro inteiro, recorta só o capítulo que a tarefa precisa. O recorte cabe em poucas fichas.

Peça	Na festa
<code>core/cognitive_ledger.py</code>	Um caderno SQLite que guarda intenção, decisão, impressão digital do arquivo e o que o guarda disse. Para a próxima pessoa não reler o chat.
<code>core/mcp_dynamic_router.py</code>	Encaminha a ligação para o telefone certo.

O guarda da escrita inteira-ou-nada é `gates/G_ESCRITOR_ATOMICO.py`. O guarda do diário é `gates/G_TRANSACTION_LOG_LRU.py`.

Capítulo 17 — O telefone — MCP — e as quatro peças que todo bolo leva

QUARTETO SINE QUA NON DINÂMICO (LEI #10)

Todo produto gerado ou evoluído no ecossistema nasce com quatro interfaces obrigatórias:

- 1. Swagger Studio ([/docs](#) e [/swagger](#)): contrato OpenAPI 3.1 da API REST;
- 2. Webhook Studio ([/webhooks](#)): hub de entrada e testes de eventos externos assíncronos;
- 3. MCP Studio ([/mcp](#)): servidor Model Context Protocol para interoperabilidade com agentes de IA;
- 4. Guia do Utilizador ([/docs/guia](#)): documentação acessível voltada a operadores humanos.

Nenhuma funcionalidade pode existir exclusivamente no MCP sem espelho na API REST (Lei 10 e Portão [G_PROTOCOL_FALLBACK.py](#)).

Na festa. O telefone da parede liga para fora: uma loja, um mapa, um gráfico da casa. O telefone não assa. Ele busca.

Também tem o contrário: o bolo pronto oferece um telefone para outros ajudantes ligarem para ele. Isso não é o mesmo aparelho.

Na casa. MCP é o protocolo desse telefone.

Dois mundos, de propósito:

- 1. Telefone de quem constrói a casa. Lista em [gates/dependencias_externas.json](#). Comando `python ecossistema.py dependencia`. Exemplo vivo: [code-review-graph](#), o mapa da casa. Antes de sair abrindo arquivo atrás de arquivo, a pessoa pergunta ao mapa: quem chama esta função, o que quebra se eu mexer aqui. Isso poupa fichas.
- 2. Telefone do programa gerado. Mora em [componentes/<escopo>/mcps/](#) e no app em [/mcp](#). Não se mistura com o telefone de quem constrói.

O mapa [code-review-graph](#) tem estas perguntas prontas: o que mudou e qual o risco, um trecho curto para revisar, o raio da mudança, os caminhos afetados, quem chama e quem é chamado, busca por nome, visão da arquitetura, plano de renomear.

Se o mapa não atender, aí sim se abre o arquivo. Não o contrário.

O quarteto

Todo programa que esta casa gera ou evolui nasce com quatro peças. Lei 10.

Peça	Endereço	Para quê
Livro da porta	/docs e o estúdio do contrato	Ver o que a porta aceita
Campinha de fora	/webhooks	O mundo avisa o programa
Telefone	/mcp	Um ajudante usa o programa
Guia	/docs/guia	Uma pessoa usa o programa

E uma regra de justiça: o que existe no telefone também existe na porta comum. Nada de função escondida só no telefone. Guarda: [gates/G_PROTOCOL_FALLBACK.py](#).

O que vigia o quarteto no bolo pronto: [gates/G_QUARTETO_SINE_QUA_NON.py](#) e [gates/G_CONTRACT_ROT.py](#).

Capítulo 18 — O envelope — handoff

Na festa. Ninguém grita a receita no corredor. Escreve, fecha, entrega. O próximo só abre. Se o envelope estiver furado, a fila para.

Na casa. Um handoff é um arquivo com forma marcada. A forma mora em [componentes/compartilhado/specs/](#). Se faltar campo, o guarda devolve.

Envelope	De quem para quem
handoff-planner-to-engine.schema.json	Da planta para a cozinha do meio (generator, factory ou bridge)
handoff-engine-to-master.schema.json	Da cozinha do meio para quem organiza as fatias
handoff-master-to-enterprise.schema.json	Das fatias para o carimbo
handoff-enterprise-to-ops.schema.json	Do carimbo para quem publica
handoff-execucao.schema.json	O envelope da fila em salas temporárias. Cada pedacinho traz título, arquivos que pode mexer, e o comando que prova que ficou certo.
vsa-topological-dispatch.schema.json	O envelope do despacho das fatias: quem depende de quem.
plano-infraestrutura.schema.json	O envelope da infra, das três primeiras etapas do ops para quem vai montar o servidor.

O envelope da conversa, diferente desses, é o [/aidd-handoff](#): um markdown curto em [docs/secoes/sessao-<data>-<slug>.md](#). Serve para trocar de pessoa sem recontar a tarde inteira. Isso é economia de ficha e também a lei 3: o estado fica no arquivo.

Guardas: [gates/G_PIPELINE_HANDOFF.py](#) para o envelope da fila, [gates/G_DISPATCH_PIPELINE_VSA.py](#) para o das fatias, [gates/G_ORQUESTRADOR_SINCRONO.py](#) para os envelopes do fluxo.

Capítulo 19 — A fila — pipeline

Na festa. Desenha, depois assa, depois cobre, depois arruma, depois carimba, depois entrega. Quem está atrás não começa no escuro. No fim tem uma barreira: ou todas as salas que tinham licença de trabalhar terminaram certo, ou ninguém vai para casa fingindo que o bolo está inteiro.

Na casa.

- `python ecossistema.py run-plan <plano>` — lê o plano, compila o envelope, corre.
- `python ecossistema.py pipeline --handoff <json>` — o envelope já existe.
- Motor: `tools/aidd-master/scripts/orchestrator_pipeline.py`.
- Cartão: `aidd-pipeline-runner`.

A barreira de junção (join barrier) é o portão do fim do corredor. Sala temporária que falha não é varrida para debaixo do tapete: a limpeza roda mesmo assim, e o estado fica marcado.

`--dry-run` mostra o que faria, sem fazer. É o ensaio.

Capítulo 20 — O despacho das fatias

ISOLAMENTO ESTRITO EM WORKTREES EFÊMERAS

O ecossistema implementa o Despacho Topológico VSA (Vertical Slice Architecture) utilizando Git Worktrees efêmeras em `.worktrees/<fatia>`. Cada fatia vertical executa isolada, validando seus próprios testes e portões. Ao término, a Join Barrier (`vsa_join_barrier.py`) valida os arquivos alterados e reconverge os artefatos para a branch principal, destruindo a worktree sob garantia estrita de `try-finally`.

Na festa. O bolo de festa não é uma massa só. Tem a lista de compras, o nome da criança, as velas. A cobertura não pode começar antes do bolo. A lista de quem vem e o cartão de feliz aniversário não dependem um do outro: podem ser feitos em salas separadas ao mesmo tempo. No fim, só entra na mesa o que cada sala tinha licença de entregar.

Na casa. Fatia vertical: um pedaço de negócio completo, de cima a baixo (dado, regra, porta, tela, teste), sem enfiar a mão na fatia vizinha. O guarda desse "não enfie a mão" é `gates/G_ISOLATION_AUDIT.py`. Se duas fatias repetirem a mesma panela, `gates/G_DRIFT_ANALYZER.py` aponta, para a panela ir para o miolo.

A ordem sai do planner. O motor é `dispatch_pipeline.py`. O roteador `engine_router.py` escolhe a cozinha do meio e cobra o quarteto. A barreira `vsa_join_barrier.py` olha `git status` e só aceita os arquivos daquela fatia. O manifesto que segue para o enterprise leva a impressão digital SHA-256.

Porta: `python ecossistema.py dispatch --planner <plano>` OU `--dispatch <json>`.

Caminho formal de uma ideia até o despacho: `/aidd-grill`, `/aidd-spec`, `/aidd-planner`, `/aidd-dispatch-runner`.

Capítulo 21 — As 8 etapas de quem asse do zero

Na festa. O generator não faz o bolo num pulo. Oito portas. Em cada porta, um teste. Se o teste volta, a próxima porta nem abre.

Na casa. `tools/aidd-generator`. Estado no `PLANO-EXECUCAO-ESTRUTURADO.json`. Passagem de fase: `python scripts/validar_fase.py --fase N` tem que passar.

Etapa	Nome	Na festa
1	Pesquisa	Olha o que já existe no mundo antes de inventar.
2	Analizador	Cruza a ideia com o que a pesquisa achou.
3	Designer	Desenha as camadas.
4	Planejador	Decide a ordem.
5	Criador	Põe a estrutura no disco, com git e caderno de verdade.
6	Documentador	Escreve o guia a partir do que foi criado, não do que foi sonhado.
7	Autocrítica	Confere fatos. Não inventa nota de investimento.
8	Implementador	Escreve o código que o teste já pediu. Sem bilhete "termino depois".

As etapas 1 e 2 desta cozinha usam spec e tickets para não cozinhar no feeling.

Capítulo 22 — O caminho de melhorar a casa que já existe

Na festa. Outra fila, para quando o bolo já está na mesa e você quer deixá-lo melhor. Não se mistura com "fazer um bolo novo".

Na casa. Um comando, um dono. Três etapas.

Etapa	Comando	Entra	Sai	Para e pergunta
1	<code>/melhoria</code>	um pedido, ou um plano para reolhar	relatório em <code>docs/melhorias/</code> com nota de 0 a 10 e a prova	"gero o plano?"
2	<code>/plan</code>	o relatório	pasta em <code>docs/planos/<nome>/</code> , ainda rascunho	"aprova?"
3	<code>/orchestrate</code>	o plano aprovado	a obra	você escolhe o ambiente e aprova o plano de voo

Os planos não ficam soltos. Moram em três caixas, e a máquina `scripts/atualizar_index_planos.py` muda o plano de caixa pelo estado real:

- `docs/planos/a-fazer/`
- `docs/planos/fazendo/`
- `docs/planos/feitos/`

`aprovar` move para `a-fazer`. `iniciar-execucao` move para `fazendo`. Nota nova exige prova escrita. Nota alvo não é nota atual. Se o plano é antigo demais para ter nota, a casa diz "não auditado". Não inventa número. Lei 8.

Onde a obra roda

`python ecossistema.py orchestrate <plano>` não executa no escuro. Ele prepara o plano de voo. Você revisa. Três ambientes:

Ambiente	Na festa
<code>orca</code>	A obra vai para o aplicativo ORCA, com mesas separadas.
<code>subagent</code>	A obra vira tarefa para um ajudante desta conversa.
<code>gitworktree</code>	A obra roda em salas git desta própria casa, sem precisar do aplicativo ORCA.

Antes de abrir a sala, a casa imprime o comando, quem vai trabalhar e qual sala. Você vê. Durante o pré-voo dá para escolher um trabalhador diferente por frente.

ORCA tem regra própria, curta: ler o manual da versão instalada antes de apertar botão; mesa independente por padrão; não retomar uma conversa velha por um número de sessão; esperar a mesa estar realmente livre antes de falar de novo; não repetir o recado no silêncio.

Capítulo 23 — Os guardas — gates

Na festa. Em cada porta, uma pessoa que não cozinha. Olha uma coisa só. Diz passa (`exit 0`) ou volta (`exit 1`). Guarda que nunca foi testado com uma mentira de propósito não é guarda: é pintura na porta. Lei 13.

Na casa. Pasta `gates/`. Chamar todos: `python ecossistema.py audit` (por baixo, o pre-commit corre nos arquivos). Cada `G_*.py` tem um `test_g_*.py` que quebra a regra de propósito.

Guarda	Olha isto
<code>G_DETERMINISMO_LEI_1</code>	Máquina mecânica chamando modelo de linguagem
<code>G_SAIDA_BINARIA</code>	Guarda saindo com outra coisa que não seja 0 ou 1
<code>G_ESTRUTURA_ESTADO</code>	Caderno de estado furado
<code>G_IDIOMA_LEI_4</code>	Texto longo demais, no lugar que devia ser curto, gastando ficha
<code>G_TESTES_REAIS</code>	Teste de verdade falhando dentro de uma cozinha
<code>G_COMPONENTE_AGNOSTICO</code>	Peça que não chegou em todas as portas
<code>G_ZERO_HEADLESS</code>	Trabalhador invisível
<code>G_HONESTIDADE_ROTULO</code>	Palavra de marketing sem prova
<code>G_DISCIPLINA_TESTE_FERRAMENTA</code>	Mudança em <code>tools/</code> sem relatório novo em <code>docs/teste-end-to-end/</code>
<code>G_QUARTETO_SINE_QUA_NON</code>	Bolo sem uma das quatro peças
<code>G_STACK_PADRAO_OURO</code>	Tela ou caderno fora do padrão, sem pedido explícito
<code>G_PORTAO_PROVA_QUE_MORDE</code>	Guarda novo sem teste que o faz dizer "volta"
<code>G_LEI_DECLARA_PORTAO</code>	Lei no cartaz sem guarda declarado
<code>G_PIPELINE_HANDOFF</code>	Envelope da fila furado
<code>G_DISPATCH_PIPELINE_VSA</code>	Ordem das fatias furada ou em círculo
<code>G_DOCS_ROT</code>	Link quebrado e papel velho no lugar do cartaz
<code>G_ORQUESTRADOR_SINCRONO</code>	Fluxo inteiro sem os envelopes
<code>G_ECOSSISTEMA_INTEGRIDADE</code>	Peça física, sintaxe e estrutura das cozinhas e dos cartões
<code>G_DRIFT_NUCLEO_COMPARTILHADO</code>	Miolo do master e do enterprise diferentes sem estar na foto
<code>G_DRIFT_ANALYZER</code>	A mesma panela copiada em duas fatias
<code>G_HARNESS_COMPAT</code>	Portas com cartazes diferentes
<code>G_UNIVERSAL_HARNESS</code>	Cartão, telefone ou campanha faltando numa porta
<code>G_SEGREDOS</code>	Senha escrita no código. A foto do que já foi olhado e descartado é <code>.secrets.baseline</code>
<code>G_CLI_HELP_CONSISTENCIA</code>	Ajuda da porta citando botão que não existe
<code>G_INFRA_COMPOSE</code>	Receita de vários containers furada ou insegura
<code>G_HADOLINT</code>	Receita de um container fora das boas práticas
<code>G_DEPENDENCIAS_PIN_HASH</code>	Pacote sem versão travada ou sem impressão digital no lock
<code>G_SUPPLY_CHAIN</code>	Cadeia do que se instala
<code>G_ARQUITETURA_DELIVERABLE</code>	Camadas do código misturadas

Guarda	Olha isto
<code>G_ESCRITOR_ATOMICO</code>	Escrita que pode deixar arquivo pela metade
<code>G_TRANSACTION_LOG_LRU</code>	Diário de transação que não é o mesmo em todos os destinos
<code>G_FRONTEND_LAYERS</code>	Tela bonita ligando sozinha para a rede, em vez de pedir para quem sabe pedir
<code>G_ISOLATION_AUDIT</code>	Fatia importando a fatia vizinha
<code>G_PROTOCOL_FALLBACK</code>	Função que só existe no telefone e não na porta
<code>G_LLM_PROMPT_SHIELD</code>	Chamada de modelo sem a blindagem do que entra
<code>G_PROTOTYPE_REWRITE</code>	Rascunho da caixa de areia promovido a casa sem teste
<code>G_LIVRO_EVIDENCIA</code>	Livro citando arquivo que não está no disco, ou escondendo o que faltou
<code>G_CONTRACT_ROT</code>	A porta de verdade diferente do contrato escrito
<code>G_ENV_ROT</code>	Código lendo botão de ambiente que o exemplo não declara
<code>G_SKILL_ROT</code>	Cartão apontando para arquivo ou comando que não existe
<code>G_MIGRATION_ROT</code>	Mudança de caderno que não vai e não volta limpa
<code>G_TEMPLATE_FORGE_ROT</code>	Molde do forge apontando para o que não existe

Como se escreve um guarda novo: [docs/protocolos/CONVENCAO-AUTORIA-GATES.md](#). Sem o teste que morde, não entra.

O ciclo da lei 9, quando a coisa que mudou é uma ferramenta, tem 5 passos. Estão em [docs/protocolos/PROTOCOLO-TESTES-FERRAMENTAS.md](#).

1. Consertar até zerar a diferença.
2. Registrar no git e enviar.
3. Limpar o projeto alvo.
4. Rodar limpo.
5. Atualizar [docs/teste-end-to-end/](#).

Capítulo 24 — As campanhas — hooks

Na festa. O guarda você vai até ele. A campanha toca sozinha, no momento certo: quando você entra, quando você tenta sair, quando você fala demais.

Na casa. Fonte: [componentes/compartilhado/hooks/](#). Cópia nas portas. Não se edita a cópia.

Campaina	Quando toca
<code>regra10_check.py</code>	Na saída da resposta. Olha a forma e o teto de fichas, com as regras de <code>regra10_termos.json</code> . O teste dela é <code>gates/test_regra10_check.py</code> .
<code>anti_headless_subagent_hook.py</code>	Quando alguém tenta soltar trabalhador invisível.
<code>crg_session_start.py</code> e os atalhos <code>.cmd</code> / <code>.sh</code>	Na entrada, para o mapa da casa estar acordado.
<code>crg_update.py</code> e os atalhos	Quando o código mudou e o mapa precisa atualizar.
<code>componentes/compartilhado/security/sast_scanner.py</code>	Varre código procurando furos conhecidos. As regras estão em <code>semgrep_rules.yml</code> .

Há também o gancho de commit (pre-commit): antes de guardar a mudança, os guardas marcados rodam. Mudança que o guarda recusou não entra no caderno do git.

Capítulo 25 — A economia das fichas

ENGENHARIA DE ATENÇÃO E TOKEN ECONOMY (LEI #4)

A economia de tokens não visa apenas redução de custos financeiros, mas principalmente a preservação da fidelidade cognitiva dos Modelos de Linguagem (LLMs). Ao manter o contexto ativo enxuto (< 2.000 tokens), consultar grafos de conhecimento antes de realizar varreduras completas e aplicar protocolos compactos (Caveman Ultra), elimina-se a alucinação e a perda de atenção em sessões prolongadas.

Na festa. A pessoa que ajuda só consegue prestar atenção numa mesa de tamanho finito. Cada palavra em cima da mesa é uma ficha. Se você põe o sótão inteiro na mesa, ela esquece a receita. A casa foi desenhada para pôr na mesa só o papel da vez.

Na casa. Ficha é token. Lei 4. Não é frescura. É o que impede a casa grande de ficar burra no meio da tarde.

O que gasta ficha à toa

- Reler o chat em vez de abrir o caderno.
- Abrir o manual comprido quando o cartaz curto bastava.
- Ler um arquivo de mil linhas para achar uma função.
- Tratar relatório velho como lei.
- Dois cartões dizendo a mesma coisa com palavras diferentes.
- Resposta longa quando a pessoa pediu um fato.
- Pensar em voz alta, em português floreado, no miolo que a máquina vai reler mil vezes.

O que a casa faz para gastar pouco

Gesto	Onde	Efeito
Cartaz curto, manual sob demanda	AGENTS.md contra docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md	A conversa comum não carrega o manual.
Memória em arquivo	MEMORY.md , PLANO-EXECUCAO-ESTRUTURADO.json , core/cognitive_ledger.py , docs/secoes/	A próxima pessoa lê uma página, não a tarde.
Recorte	core/context_slicer.py	Entra o capítulo, não o livro. O recorte cabe em cerca de 150 fichas.
Mapa antes da leitura	code-review-graph	Pergunta "quem chama isto" em vez de abrir tudo.
Fala comprimida no miolo	caveman_protocol.py e cartões escritos em inglês curto	O que a máquina relê o tempo todo é denso. A resposta para a pessoa continua em português. O próprio protocolo descreve corte de 30% a 50% no que entra.
Teto da resposta	regra10_termos.json	300 fichas no normal, 600 no técnico, a não ser que a pessoa peça o mapa.

Gesto	Onde	Efeito
Pensamento curto	a lei, no cartaz	Poucas linhas, em inglês compacto, só o que muda a decisão.
Estado entre fases	JSON, não história	O generator lê o caderno da fase. Não a conversa.
Envelope	os schemas em componentes/compartilhado/specs/	A fase seguinte recebe o contrato, não o diário.
Uma fonte só	componentes/	Sete portas não repetem sete textos diferentes na mesa.
Não ingerir o sótão	lei 12, G_DOCS_ROT.py	Papel velho não vira contexto.
Purge entre fases	o forge e o fluxo	O que a fase anterior já entregou não viaja como romance.
Livro sem modelo	python ecossistema.py livro	O livro de evidência é montado por máquina, a partir do disco. Zero ficha de modelo. O texto explicativo, se você quiser, é outro passo: /aidd-livro-texto .

Como uma pessoa deve falar com a casa

- Peça uma coisa.
- Aponte o cômodo, se souber.
- Não cole o sótão.
- Se quiser o mapa, diga que quer o mapa. Aí o teto curto não vale, porque você pediu o contrário.
- Se a resposta veio grande sem você pedir, a campainha deveria ter tocado.

O que não é economia

Economia não é esconder passo, não é dizer "pronto" sem prova, não é cortar o teste. A resposta fica curta. O trabalho fica inteiro. O código que sai continua completo, tipado, com teste real. A compressão é da conversa, não do bolo.

Capítulo 26 — Como se monta um projeto deste tamanho

Na festa. Uma casa deste tamanho não nasce de um cozinheiro heroico. Nasce de gaveta única, cópia nas portas, lei na parede, guarda em cada porta, envelope entre as salas, e um caderno que não depende de ninguém lembrar.

Na casa. A composição, na ordem em que você montaria de novo.

1. **Uma fonte.** Tudo que é cartão, comando, telefone, campanha ou contrato nasce em `componentes/<escopo>/<tipo>/`. Escopo `compartilhado` vale para a casa toda. Escopo de uma cozinha vale só para ela.
2. **Um manifesto de cópia.** `gates/manifesto_harnesses.json` diz as sete portas e o formato de cada uma. Você não copia na mão.
3. **Sete portas, um texto.** Claude (`.claude`), Antigravity (`.agents`), OpenCode (`.opencode`), MiMo (`.mimocode`), Gemini (`.gemini/extensions`), Cursor (`.cursor`), CodeBuddy (`.codebuddy`). `python ecossistema.py components sync` materializa. `verify` confere a impressão digital.
4. **Um cartaz e uma memória.** `AGENTS.md` é lei. `MEMORY.md` é o que a casa já decidiu. Ponteiros nas portas não repetem a lei.
5. **Botões em arquivo.** Tudo que é número, lista, exceção ou teto está nos JSON da seção 6. Mudou o botão, mudou o arquivo. Não mudou "o combinado de hoje".
6. **Oito cozinhas.** `tools/aidd-*`, cada uma com o seu `AGENTS.md` local para o detalhe daquela cozinha. O cartaz da raiz continua mandando.
7. **Uma porta.** `ecossistema.py` é o único jeito oficial de atravessar. Atalho de voz chama a mesma porta.
8. **Contratos no meio.** Nenhuma cozinha empurra pasta mágica para a próxima. Empurra envelope com forma em `componentes/compartilhado/specs/`.
9. **Guardas com dente.** Cada lei aponta um `gates/G_*.py`, e cada guarda aponta um teste que o faz falhar.
10. **Campainhas nos momentos.** Entrada, saída, commit. Não no meio, inventadas.
11. **Estado fora da cabeça.** Plano, ledger, seções, JSON de execução, git.
12. **O bolo gerado repete a casa em pequeno.** Quatro peças, tela padrão, caderno padrão, contrato único, fatias que não se importam, teste de verdade.

O que nasce dentro do programa gerado

Peça do programa	De onde a casa tira
Tela Next.js	padrão-ouro, exportada do contrato
Miolo Python	a cozinha do meio
Caderno SQLite WAL	padrão-ouro, salvo pedido contrário na planta
<code>/docs, /swagger</code>	OpenAPI 3.1
<code>/webhooks</code>	estúdio de avisos
<code>/mcp</code>	estúdio do telefone do produto
<code>/docs/guia</code>	guia de quem usa
Fatias	master, uma pasta de negócio por vez (<code>src/modules/<modulo>/</code> no master)

Peça do programa	De onde a casa tira
Carimbo	enterprise, SHA-256
Servidor	ops: tamanho da máquina, Docker, segredo fora do código (<code>sops</code> + <code>age</code>), vigia de "está no ar"

Segredo não mora na receita. O exemplo do que existe (`.env.example`) declara os nomes. O valor fica de fora. Guarda: `G_ENV_ROT.py` e `G_SEGREDOS.py`.

Caixa de areia não vira casa. `G_PROTOTYPE_REWRITE.py` impede promover rascunho sem a suíte de testes correspondente.

Três jeitos de uma obra andar, sem confundir

Você quer	Fila
Um programa novo	fluxo <code>/pure</code> , <code>/open</code> ou <code>/freedom</code>
Uma fatia nova dentro da ordem da planta	<code>/dispatch</code>
Um plano de tickets já escrito, em salas temporárias	<code>/run-plan</code> ou <code>/pipeline</code>
Melhorar o que já está de pé	<code>/melhoria</code> , depois <code>/plan</code> , depois <code>/orchestrate</code>

São quatro portas. Misturar o nome delas é o jeito mais rápido de assar o bolo na sala errada.

Capítulo 27 — O sótão — o que não é cartaz

Na festa. A casa tem sótão. Lá tem diário, tentativa, relatório, plano antigo. Serve para lembrar um concerto. Não serve para dizer qual é a lei de hoje.

Na casa. Lei 12. Verdade viva:

- AGENTS.md
- MEMORY.md
- docs/protocolos/
- os contratos em componentes/compartilhado/specs/
- o código que os guardas olham

Sótão, útil e não-lei:

Pasta	O que é
docs/explicacoes/	Histórias como esta, para gente entender.
docs/planos/	Obra combinada, nas três caixas.
docs/melhorias/	Nota com prova, antes do plano.
docs/secoes/	Envelope de uma conversa que trocou de pessoa.
docs/teste-end-to-end/	O relatório do ciclo de 5 passos.
docs/issues/	Os pedacinhos de uma obra já cortada.
docs/livros/	O livro da casa.
docs/relatorios/, docs/reports/, docs/oficiais/, docs/manuais/, docs/features/	Papel gerado. Lê se a tarefa for aquele papel. Não carrega como lei.
docs/padroes/	Notas de padrão que não substituem docs/protocolos/.
projetos/	Um lugar de prova, não o produto.

Se este arquivo e o cartaz discordarem, manda o cartaz.

Capítulo 28 — Segurança, sem fantasia

Na festa. Fechadura, crachá, carimbo, "não deixe a chave em cima da mesa", "não diga que a porta é de banco se ela é de madeira".

Na casa. O que existe de verdade, sem adjetivo de marketing:

- Varredura de segredo no que o git acompanha: [G_SEGREDOS.py](#), foto em [.secrets.baseline](#).
- Varredura de código: [sast_scanner.py](#) e [semgrep_rules.yml](#).
- Blindagem do que entra num modelo: [G_LLM_PROMPT_SHIELD.py](#).
- Pacote com versão e impressão digital: [G_DEPENDENCIAS_PIN_HASH.py](#), [G_SUPPLY_CHAIN.py](#).
- Receita de container: [G_HADOLINT.py](#), [G_INFRA_COMPOSE.py](#).
- Carimbo de pacote: [assinatura_manifesto.py](#), chave pública em [chaves/manifesto/ed25519_public.json](#).
- Enterprise confere SHA-256 do que injeta. Isso é conferir impressão digital. Não é, sozinho, um certificado de que o bolo é seguro contra tudo.
- Ops prevê segredo lacrado ([sops](#) + [age](#)) e um vigia de disponibilidade. O detalhe daquela cozinha mora em [tools/aidd-ops/AGENTS.md](#), não neste mapa.

Lei 8 fica de sentinela nesta seção: o nome do guarda não é a prova. A prova é o teste que o faz dizer "volta".

Capítulo 29 — Glossário — uma frase cada

Palavra	Na festa
Ecosistema	A casa inteira.
Monorepo	Um depósito só, com várias cozinhas.
AGENTS.md	O cartaz da lei.
MEMORY.md	O caderno do que a casa já é.
Rule	Bilhete pequeno em volta do cartaz.
Config	Botão marcado em arquivo.
Lei	Frase que não se negocia, com guarda.
Spec	A receita colada na geladeira.
Grill	A entrevista antes da receita.
Ticket	Um pedacinho da receita, com prova.
TDD	Testar antes de fazer. Vermelho, verde, arrumar.
Planner	Quem desenha a planta.
PLANNER.json	A planta.
Workflow / fluxo	Qual das três cozinhas abre.
Tríade	As três cozinhas: pure, open, freedom.
Agent	Uma pessoa, um trabalho.
Skill	O cartão que a pessoa lê.
Command	O que você fala. Tem um cartão dono.
CLI	A porta da rua, ecossistema.py .
Script	A máquina que faz sempre igual.
MCP	O telefone. De quem constrói, ou do programa pronto. São dois.
Quarteto	Livro, campanha de fora, telefone, guia.
Handoff	O envelope fechado entre duas pessoas.
Schema	A forma do envelope. Se não cabe, volta.
Pipeline	A fila. O próximo espera o anterior.
Worktree	A sala temporária, jogada fora no fim.
Join barrier	O portão do fim: ou todas as salas certas terminaram, ou ninguém finge que acabou.
Dispatch	Mandar cada fatia para a sua sala, na ordem de quem depende de quem.
Fatia / VSA	Um pedaço de negócio inteiro, que não mexe no pedaço vizinho.
Gate	O guarda. Passa ou volta.
Hook	A campanha que toca sozinha.
Harness	O programa de conversa: Claude, Cursor, Gemini, OpenCode, MiMo, Antigravity, CodeBuddy.
Sync	Copiar a gaveta única para as portas.
Token / ficha	Uma unidade de atenção. Acabou a mesa, a pessoa esquece.

Palavra	Na festa
Caveman	Falar denso no miolo para caber mais trabalho na mesma mesa.
Ledger	O caderninho que lembra a sessão sem reler o chat.
Context slicer	O recorte do capítulo, em vez do livro.
Audit	Chamar os guardas todos.
Dry-run	Ensaiai sem fazer.
Preflight	Olhar se a máquina tem as ferramentas antes de começar.
Forge	Montar cartaz e guarda num projeto novo.
Master	Organizar as fatias.
Enterprise	Carimbar e conferir o carimbo.
Ops	Publicar num computador que fica ligado.
Factory	Encaixar pedaço pronto.
Bridge	Tirar o cadeado de um bolo feito em outra ferramenta.
Generator	Assar do zero, em 8 etapas.
ORCA	O aplicativo de mesas separadas para uma obra.
Plano de voo	O papel que você aprova antes da obra começar.
Nota	Um número de 0 a 10 que só vale com a prova do lado.
Sótlão	Papel útil que não é lei.
Padrão-ouro	A tela, o miolo e o caderno que valem quando você não pediu outros.
SHA-256	A impressão digital de um arquivo. Se um byte muda, a digital muda.
Exit 0 / exit 1	As duas únicas frases do guarda: passa, volta.
Headless	Trabalhador no escuro. Proibido no normal.
Stub	Bilhete "termino depois" no lugar do pedaço. Proibido.
WAL	O caderno aguenta várias escritas sem rasgar.
OpenAPI	A lista do que a porta aceita, escrita para máquina e para gente.

Capítulo 30 — Mapa de pastas que importam

ecossistema.py	porta da rua
AGENTS.md	cartaz
MEMORY.md	memória
CLAUDE.md GEMINI.md ...	ponteiros
PLANO-EXECUCAO-ESTRUTURADO.json	caderno de execução
requirements.txt requirements.lock	o que se instala
pytest.ini	como o teste sobe
componentes/compartilhado/comandos/	comandos de voz, fonte
componentes/compartilhado/skills/	cartões, fonte
componentes/compartilhado/hooks/	campanhas, fonte
componentes/compartilhado/specs/	forma dos envelopes
componentes/compartilhado/src-core/	panelas de todas as cozinhas
componentes/compartilhado/security/	varredura de furos
gates/G_*.py	guardas
gates/test_g_*.py	o teste que faz o guarda morder
gates/manifesto_harnesses.json	como copiar para as portas
gates/dependencias_externas.json	o que vem de fora
scripts/	máquinas da raiz
core/	recorte, ledger, roteador de telefone
tools/aidd-forge ... tools/aidd-ops	as oito cozinhas
tools/aidd-master/scripts/	fila, despacho, barreira, roteador
docs/protocolos/	lei comprida, viva
docs/explicacoes/	histórias como esta
docs/planos/	obra: a-fazer, fazendo, feitos
docs/melhorias/	nota com prova
docs/secoes/	envelope de conversa
docs/teste-end-to-end/	prova do ciclo de ferramenta
.claude .agents .cursor .opencode	cópias. não editar.
.mimocode .gemini .codebuddy	cópias. não editar.

Capítulo 31 — A mesma tarde, agora com o app de lista de compras

Você diz: "quero um app de lista de compras, do zero, para a família não esquecer o leite".

1. O cartaz já está lido. A casa não vai inventar outra tela, porque você não pediu.
2. O grill pergunta: a lista é de uma pessoa ou da família? Some quando fecha o celular? Quem risca pode desfazer? Sem internet, anota mesmo assim?
3. A spec cola na geladeira as respostas que deram sim ou não.
4. Os tickets cortam: "acrescentar item", "riscar", "continuar lá depois de fechar". Cada um com o teste que prova.
5. O planner desenha a planta e a ordem. A fatia "lista" não depende da fatia "nome da família" para existir, então podem ir a salas diferentes. A fatia "compartilhar" espera as duas.
6. Você escolhe o fluxo [/pure](#).
7. O forge garante cartaz e guardas no projeto novo. O envelope sai do planner para o generator.
8. O generator passa nas 8 portas. Estado no caderno JSON, não no chat.
9. O master recebe o envelope e organiza as fatias. O dispatch, se a planta pedir, manda cada fatia a uma sala temporária.
10. Cada sala só mexe nos arquivos dela. A barreira confere. Sala é apagada.
11. O enterprise carimba. O ops prepara o computador que fica ligado, sem colar senha na receita.
12. O programa nasce com [/docs](#), [/webhooks](#), [/mcp](#), [/docs/guia](#), tela Next.js, miolo Python, caderno SQLite.
13. Se a conversa estiver longa, o handoff escreve uma página em [docs/secoes/](#) e a próxima pessoa começa dali.
14. Se no meio alguém responder um romance sem você pedir, a campainha da regra 10 toca.
15. Se um guarda vir amendoim onde a receita proibiu, a frase é uma só: volta.

Essa é a casa. Não é mágica. É despensa etiquetada, fila, envelope e porta que só abre para os dois lados: passa, ou volta.

Capítulo 32 — Matriz Canônica de Rastreabilidade e Evidências

GARANTIA DE EVIDÊNCIA FÍSICA (LEI #2 E LEI #8)

Conforme a Lei 8 (Honestidade de Rótulo) e a convenção de livros corporativos AIDD, nenhuma afirmação técnica sobre o ecossistema é mantida sem a indicação explícita do arquivo físico, teste automatizado ou esquema de contrato que a sustenta no repositório. A matriz a seguir mapeia cada elemento do livro ao seu artefato vivo correspondente.

Elemento do Ecossistema	Artefato Canônico de Código / Governança	Portão de Validação / Teste
Porta de Entrada Unificada	<code>ecossistema.py</code>	<code>gates/G_CLI_HELP_CONSISTENCIA.py</code>
Cartaz Supremo de Leis	<code>AGENTS.md</code>	<code>gates/G_LEI_DECLARA_PORTAO.py</code>
Memória Contínua	<code>MEMORY.md</code>	<code>gates/G_DOCS_ROT.py</code>
Referência Completa	<code>docs/protocolos/AGENTS-REFERENCIA-COMPLETA.md</code>	<code>gates/G_DOCS_ROT.py</code>
Padrão-Ouro de Stack	<code>docs/protocolos/PADRAO-OURO-STACK-TECNOLOGICA.md</code>	<code>gates/G_STACK_PADRAO_OURO.py</code>
Manifesto Multi-Harness	<code>gates/manifesto_harnesses.json</code>	<code>gates/G_UNIVERSAL_HARNESS.py</code>
Sincronizador de Componentes	<code>scripts/gestor_componentes.py</code>	<code>gates/G_COMPONENTE_AGNOSTICO.py</code>
Tríade Canônica (Fluxos)	<code>scripts/orquestrador_sincrono.py</code>	<code>gates/G_ORQUESTRADOR_SINCRONO.py</code>
Despacho Topológico VSA	<code>tools/aidd-master/scripts/dispatch_pipeline.py</code>	<code>gates/G_DISPATCH_PIPELINE_VSA.py</code>
Barreira de Junção Git	<code>tools/aidd-master/scripts/vsa_join_barrier.py</code>	<code>gates/G_ISOLATION_AUDIT.py</code>
Roteador de Motores	<code>tools/aidd-master/scripts/engine_router.py</code>	<code>gates/G_QUARTETO_SINE_QUANON.py</code>
Pipeline em Worktrees	<code>tools/aidd-master/scripts/orchestrator_pipeline.py</code>	<code>gates/G_PIPELINE_HANDOFF.py</code>
Compilador de Tickets	<code>scripts/compilador_tickets_plano.py</code>	<code>gates/G_DETERMINISMO_LEI_1.py</code>
Campaina da Regra 10	<code>componentes/compartilhado/hooks/regra10_check.py</code>	<code>gates/test_regra10_check.py</code>
Bloqueio Anti-Headless	<code>componentes/compartilhado/hooks/anti_headless_subagent_hook.py</code>	<code>gates/G_ZERO_HEADLESS.py</code>
Auditoria Determinística	<code>gates/G_*.py</code>	<code>gates/G_PORTAO_PROVA_QUE_MORDE.py</code>
Protocolo de Teste de Ferramentas	<code>docs/protocolos/PROTOCOLO-TESTES-FERRAMENTAS.md</code>	<code>gates/G_DISCIPLINA_TESTE_FERRAMENTA.py</code>
Gestão de Planos e Melhorias	<code>scripts/gerenciador_planos.py</code>	<code>gates/G ESTRUTURA_ESTADO.py</code>
Protocolo Caveman Ultra	<code>componentes/compartilhado/src-core/caveman_protocol.py</code>	<code>gates/G_IDIOMA_LEI_4.py</code>

Elemento do Ecossistema	Artefato Canônico de Código / Governança	Portão de Validação / Teste
Escrita Atômica Transacional	<code>componentes/compartilhado/src-core/escritor_atomico.py</code>	<code>gates/G_ESCRITOR_ATOMICO.py</code>
Catálogo de Dependências	<code>gates/dependencias_externas.json</code>	<code>gates/G_DEPENDENCIAS_PIN_HASH.py</code>
Base do Núcleo Compartilhado	<code>gates/baseline_nucleo_compartilhado.json</code>	<code>gates/G_DRIFT_NUCLEO_COMPARTILHADO.py</code>